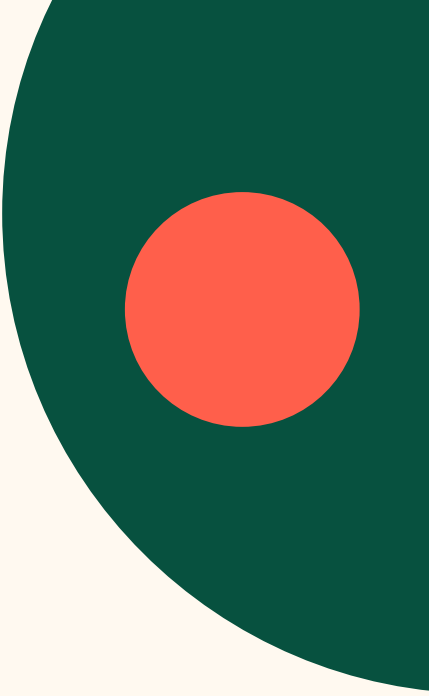




PANTRY-TO-PLATE / TEAM LEARNING GUIDE

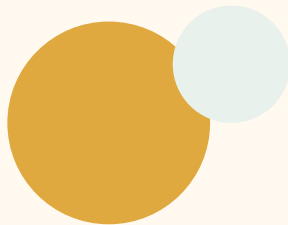
# How Our Backend Works

A beginner-friendly guide to domains, mapping, request flow, database relationships, and file ownership.

|         |         |           |                |
|---------|---------|-----------|----------------|
| DOMAINS | MAPPING | DATA FLOW | FILE OWNERSHIP |
|---------|---------|-----------|----------------|

Page asks. Controller receives. Service decides. Prisma stores. Database remembers.

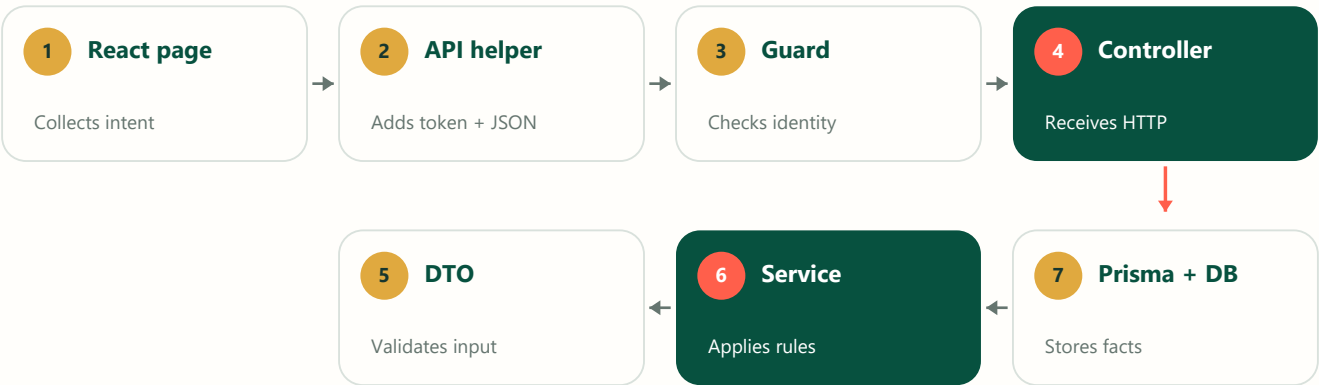
PLATESENSE FOOD ENGINE / TEAM EDITION / JULY 2026



01 / THE BIG PICTURE

# The backend is the product's decision engine

The frontend shows the experience. The backend protects identity, applies Pantry-to-Plate rules, stores facts, and returns trusted results.



The response returns through the same path in reverse.

**The sentence to remember**

Page asks, controller receives, service decides, Prisma stores, database remembers.

## What the backend is responsible for

|                                                                                               |                                                                                             |
|-----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| <b>Receive</b><br>Accept requests from the React application through versioned API endpoints. | <b>Protect</b><br>Verify the user, enforce roles, rate limits, and safe input validation.   |
| <b>Decide</b><br>Apply inventory, recipe, shopping, cooking, and nutrition rules.             | <b>Remember</b><br>Read and write persistent records in Supabase PostgreSQL through Prisma. |

## 02 / CORE CONCEPTS

# Domain tells us who owns the rule. Mapping tells us how data moves.

Start with the business language before introducing framework vocabulary. This helps new backend developers understand why a file exists.

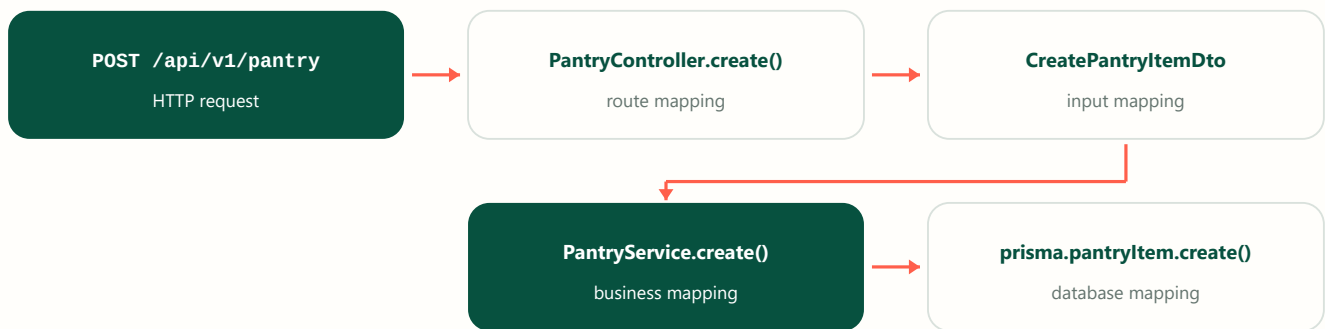
**DOMAIN**

One business area of the product: Pantry, Recipes, Meal Planner, Shopping List, Cooking Mode, Nutrition, Users, or Auth.

**MAPPING**

The connection between a request, a function, an input object, a business action, and a stored database record.

## One request, five mappings

**Use business questions first**

Ask: Which domain owns this rule? Then ask: Is the change about input, behavior, HTTP routing, or persistent storage?

## Three mappings you will see every day

**HTTP mapping**

A URL and method map to a controller function.

**Object mapping**

Request JSON maps to a validated DTO and service input.

**Database mapping**

Prisma models map TypeScript objects to PostgreSQL tables and relations.

03 / PROJECT ANATOMY

# The folder tree mirrors the responsibility of each layer

The frontend and backend live in the same repository, but they have different jobs and separate dependency boundaries.

```
pantry-inventory/  
  src/ React frontend  
  pages/ User-facing screens  
  services/api.ts Shared API + auth helper  
  types/inventory.ts Frontend response contracts  
  
backend/  
  src/main.ts Starts and configures the API  
  src/app.module.ts Registers every backend domain  
  src/common/ Guards, decorators, filters, utilities  
  src/modules/ Business domains  
  src/prisma/ Shared database connection  
  prisma/schema.prisma Current data model  
  prisma/migrations/ Database change history  
  
vercel.json Routes /api to the backend
```

## The five files inside a normal domain

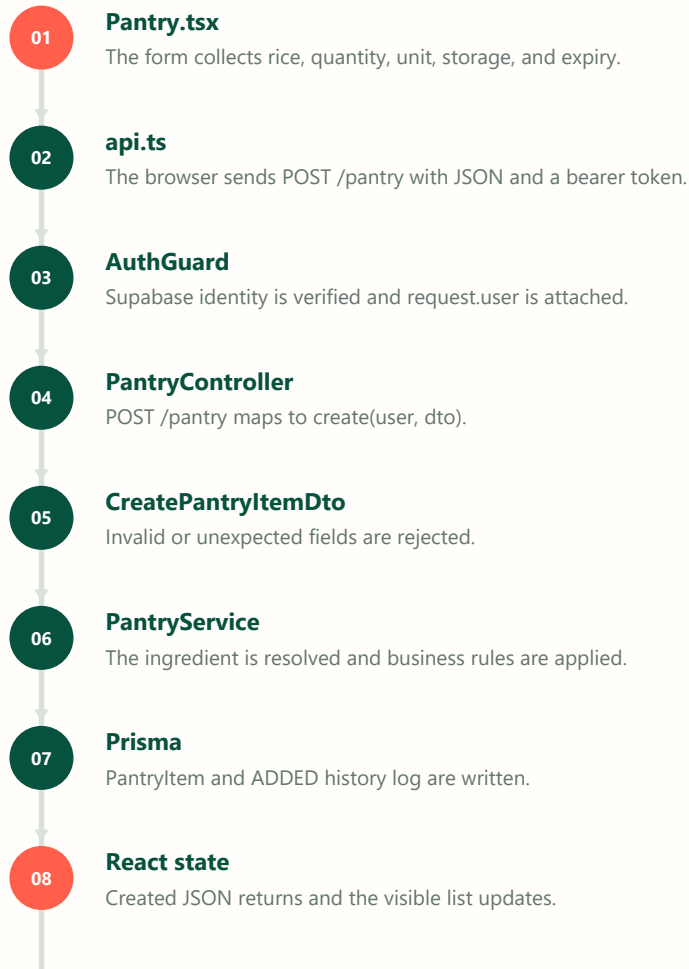
| FILE            | RESPONSIBILITY    | PLAIN MEANING                                  |
|-----------------|-------------------|------------------------------------------------|
| *.module.ts     | Dependency wiring | Connects the domain's parts.                   |
| *.controller.ts | HTTP routes       | Receives requests and returns responses.       |
| *.service.ts    | Business rules    | Decides what should happen.                    |
| dto/*.dto.ts    | Input contract    | Defines and validates what clients may send.   |
| schema.prisma   | Stored model      | Defines fields, relations, and database shape. |

**Layer rule**  
Controllers should stay thin. Services own business decisions. Prisma handles stored data.

## 04 / REAL REQUEST WALKTHROUGH

# What happens when a user adds rice to Pantry?

Following one real request is the fastest way to connect the abstract layers to actual project files.

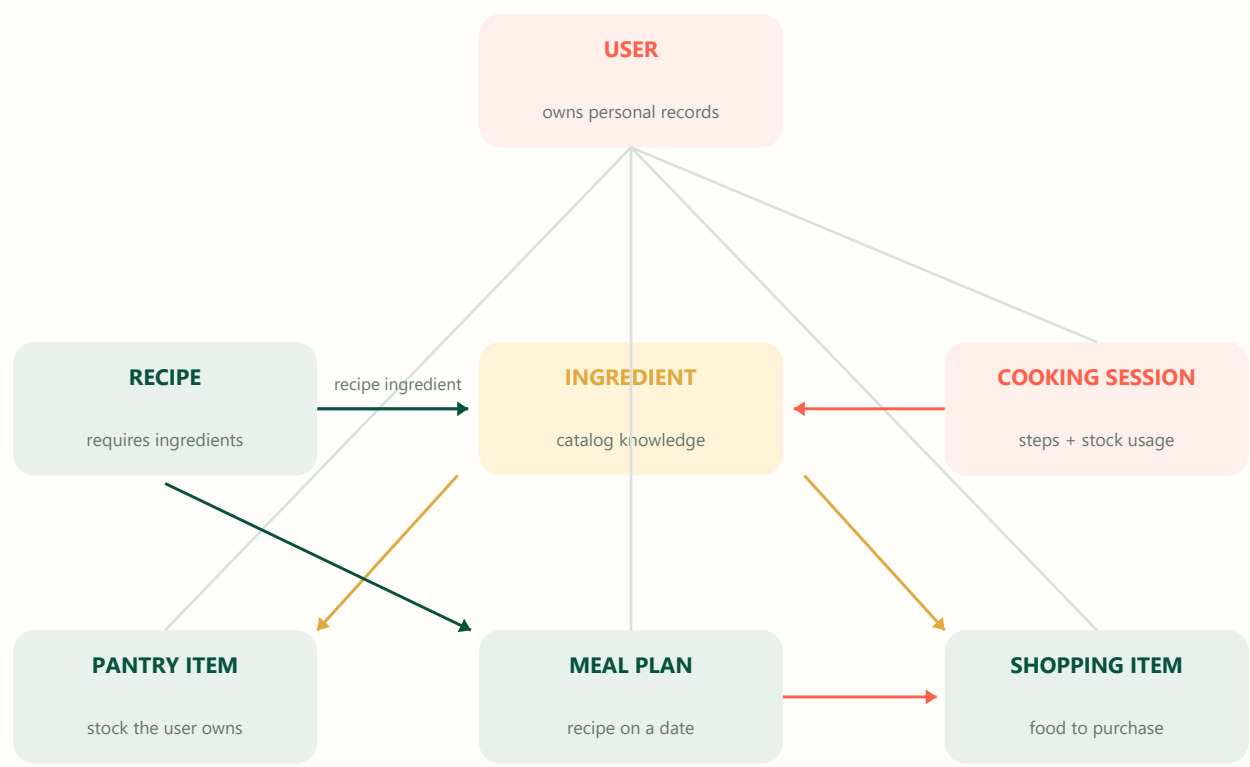
**The request is user-scoped**

The controller passes user.id into PantryService. Every user-owned query must include userId so one account cannot read or change another account's pantry.

05 / DATA MODEL

# Similar words can represent different business facts

Rice can be catalog knowledge, owned stock, a recipe requirement, or a shopping intention. The data model keeps those meanings separate and relates them safely.



Relations connect business facts; they are not duplicate copies of the same object.

| MODEL            | WHAT IT MEANS            | EXAMPLE                         |
|------------------|--------------------------|---------------------------------|
| Ingredient       | Shared catalog knowledge | Rice exists; default unit is g. |
| PantryItem       | Stock owned by one user  | Ada has 2 kg of rice.           |
| RecipeIngredient | A recipe requirement     | Jollof needs 500 g of rice.     |
| ShoppingListItem | An intention or purchase | Buy 1 kg of rice.               |

06 / DOMAIN OWNERSHIP

# Every major business area has a clear backend home

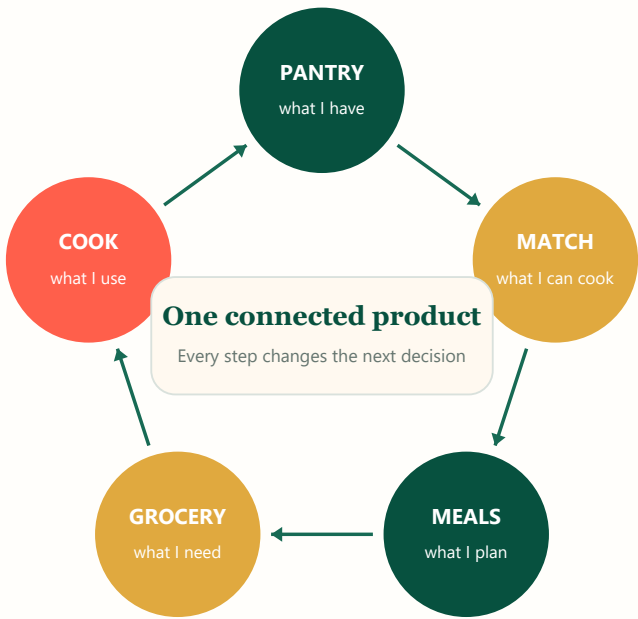
Use this map when deciding where a new endpoint, validation rule, calculation, or database action belongs. Modules can consume other domains through explicit imports and exported services.

|                                                                              |                                                                                   |
|------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <b>Auth</b><br>Identity and sessions<br>auth.controller.ts / auth.service.ts | <b>Users</b><br>Profile and preferences<br>users.controller.ts / users.service.ts |
| <b>Ingredients</b><br>Catalog, aliases, nutrition<br>ingredients/*           | <b>Pantry</b><br>Owned stock and history<br>pantry/*                              |
| <b>Recipes</b><br>Meals, requirements, steps<br>recipes/*                    | <b>Recipe Matcher</b><br>Pantry versus requirements<br>recipe-matcher/*           |
| <b>Meal Planner</b><br>Dated meal slots and AI preview<br>meal-planner/*     | <b>Shopping List</b><br>Missing food and purchases<br>shopping-list/*             |
| <b>Cooking Mode</b><br>Sessions, steps, stock usage<br>cooking-mode/*        | <b>Nutrition</b><br>Estimates, logs, summaries<br>nutrition/*                     |
| <b>Dashboard</b><br>Cross-domain summary<br>dashboard/*                      | <b>Recommendations</b><br>Pantry-aware suggestions<br>recommendations/*           |
| <b>Favorites</b><br>Saved recipes<br>favorites/*                             | <b>Food Analysis</b><br>Gemini image analysis<br>food-analysis/*                  |
| <b>Measurements</b><br>User conversion profiles<br>measurement-profiles/*    | <b>Admin</b><br>Catalog and moderation tools<br>admin/*                           |

07 / CONNECTED PRODUCT FLOW

# Pantry, Meals, Grocery, and Cooking are one inventory loop

These domains are separate for ownership, but their business outcomes must remain connected and transactionally safe.



|                                                                                                                           |                                                                                                                           |
|---------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>SHOPPING COMPLETION</b><br>Bought ingredients are added into Pantry and recorded with a BOUGHT history log.            | <b>COOKING COMPLETION</b><br>Used ingredients are deducted from Pantry. Completion is blocked when stock is insufficient. |
| <b>SHOPPING GENERATION</b><br>Meal requirements are compared against Pantry so the list contains only missing quantities. | <b>NUTRITION RESULT</b><br>A completed cooking session can create a nutrition log for later summaries.                    |

**Why transactions matter**  
When several records must change together, a database transaction makes every write succeed or every write fail. Partial inventory updates would corrupt the product story.



08 / SHARED INFRASTRUCTURE

# Some files influence every domain

These cross-cutting layers make the API consistent before and after each domain-specific business action.

| FILE                         | WHY IT MATTERS                                                         |
|------------------------------|------------------------------------------------------------------------|
| backend/src/app.module.ts    | Registers modules and global guards.                                   |
| backend/src/main.ts          | Sets API prefix, validation, CORS, security, logging, and Swagger.     |
| backend/src/common/guards/   | Checks authentication, roles, and rate limits before controllers run.  |
| backend/src/common/utils/    | Shares date, string, and unit conversion rules.                        |
| backend/src/prisma/          | Provides one shared Prisma database connection.                        |
| backend/prisma/schema.prisma | Defines the current database structure and relations.                  |
| src/services/api.ts          | Centralizes frontend authentication, JSON, errors, retry, and caching. |
| vercel.json                  | Routes production /api traffic to the backend service.                 |

## Eight safe-backend habits

|                                                                                                 |                                                                                         |
|-------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>1. Validate every request</b><br>The frontend is helpful, but the DTO is the trust boundary. | <b>2. Scope by userId</b><br>Never read or change another user's records.               |
| <b>3. Keep rules in services</b><br>Do not hide product logic in React pages or controllers.    | <b>4. Use transactions</b><br>Coupled writes must succeed or fail together.             |
| <b>5. Migrate stored changes</b><br>schema.prisma changes need a migration.                     | <b>6. Protect secrets</b><br>Gemini and database credentials stay server-side.          |
| <b>7. Align contracts</b><br>Frontend types must match backend responses.                       | <b>8. Test unhappy paths</b><br>Invalid input and missing stock are expected scenarios. |

09 / FINDING THE RIGHT FILE

# Use three questions before touching code

A good backend change begins with ownership, then narrows to the layer, then checks every consumer affected by the rule.

|                                                                                                                                                     |                                                                                                                                                 |                                                                                                                                         |
|-----------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>1 / WHO OWNS THE RULE?</b><br><br>Stock quantity belongs to Pantry. What to buy belongs to Shopping List. Recipe requirements belong to Recipes. | <b>2 / WHAT KIND OF CHANGE?</b><br><br>Input means DTO. Behavior means service. URL means controller. Stored field means schema plus migration. | <b>3 / WHO CONSUMES IT?</b><br><br>A low-stock change can affect Pantry, Dashboard, Recommendations, Shopping List, and frontend types. |
|-----------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|

## Change-to-file examples

| REQUESTED CHANGE         | START HERE                        | THEN CHECK                           |
|--------------------------|-----------------------------------|--------------------------------------|
| Add a supplier field     | Pantry DTO/service + schema       | Migration, frontend type and form    |
| Change low-stock logic   | pantry.service.ts                 | Dashboard and recommendations        |
| Add a Pantry endpoint    | pantry.controller.ts              | DTO, service, tests                  |
| Change missing groceries | shopping-list.service.ts          | Conversions and measurement profiles |
| Change cooking deduction | cooking-mode.service.ts           | Pantry logs and transaction          |
| Add a backend domain     | New module/controller/service/DTO | Register in app.module.ts            |

**Do not start from the filename**  
Start from the business rule. The correct filename becomes obvious once ownership and layer are clear.

## 10 / TEACH IT TOGETHER

# A practical 45-minute backend learning session

The team should leave able to trace a request, identify the owner of a rule, and name the correct layer for a change.

- 
- 00-05** **Tell the product story**  
Pantry to Recipes to Meals to Grocery to Cooking.
  - 05-15** **Draw the pipeline**  
Page, API helper, Guard, Controller, DTO, Service, Prisma, DB.
  - 15-30** **Trace one real request**  
Open the six Pantry files and follow POST /pantry.
  - 30-40** **Team exercise**  
Map a new 'purchased from' field without coding it.
  - 40-45** **Review**  
Each person explains one layer in plain language.

## Open these files side by side

1. `src/pages/dashboard/Pantry.tsx`
2. `src/services/api.ts`
3. `backend/src/modules/pantry/pantry.controller.ts`
4. `backend/src/modules/pantry/dto/pantry.dto.ts`
5. `backend/src/modules/pantry/pantry.service.ts`
6. `backend/prisma/schema.prisma`

### Exercise

Map this change without coding it: 'We want to record where each pantry item was purchased.' Identify schema, migration, DTO, service, frontend, authorization, and tests.

## 11 / QUICK REFERENCE

# Backend words translated into plain language

Use the project vocabulary consistently. Clear names make it easier to discuss ownership, defects, and future changes.

|                                                                       |                                                                        |
|-----------------------------------------------------------------------|------------------------------------------------------------------------|
| <b>API</b><br>The agreement for how frontend asks backend for work.   | <b>Endpoint</b><br>One method and URL, such as POST /api/v1/pantry.    |
| <b>Controller</b><br>The receptionist that receives an HTTP request.  | <b>DTO</b><br>The allowed and validated shape of input data.           |
| <b>Service</b><br>The place where business decisions are made.        | <b>Module</b><br>The wiring that groups one domain's parts.            |
| <b>Guard</b><br>A check that runs before a controller.                | <b>ORM</b><br>A tool that represents database records as code objects. |
| <b>Prisma</b><br>The ORM used by Pantry-to-Plate.                     | <b>Model</b><br>A stored business object and its relationships.        |
| <b>Migration</b><br>A versioned change to database structure.         | <b>Relation</b><br>A connection between stored models.                 |
| <b>Transaction</b><br>Database changes that succeed or fail together. | <b>Authentication</b><br>Proving who a user is.                        |
| <b>Authorization</b><br>Deciding what that user may do.               | <b>Swagger</b><br>Browsable documentation for available API routes.    |

**Useful local tools**

Swagger: <http://localhost:4000/api/v1/docs> | Prisma Studio: run `pnpm prisma:studio` inside backend/

Start from the business domain, trace the request through the layers, and keep each rule in the layer that owns it.